JAVA / 阶段 01

面向对象程序设计

阶段一 · 零基础详解

对应原题库全部 30 道题

从基础语法到独立完成程序

使用方式

每道题保存为独立的 Main.java。先读题并自己尝试，再对照答案运行；不要把不同题的 Main 类放进同一个文件。

命令：javac -encoding UTF-8 Main.java；随后执行 java Main。输入样例中的数据后按回车。网站可直接编辑、编译和运行。

解法使用变量、判断、循环、数组、方法和基础面向对象知识，不依赖第三方 Java 库。

题目口径与说明

1. 依据用户提供的阶段一题库逐题整理，共30个原编号；每题是独立、无包名的 Main.java，使用JDK17基础语法。
2. 代码为学习练习参考，建议先独立尝试再对照。样例中的标准输入不包含解释文字，固定数据题输入为空。
3. 第14与15条在原文中属于同一题及其示例，为对应原编号保留两条；统一输入顺序为 n a。
4. 第24题修正了原文不合法的数组初始化语法及中文逗号；第27题兼答括注的最小值，行列号从1开始。
5. 第10题不一定存在三位约数，已补充不存在的输出；第19题空数据平均值按程序约定显示0.0；第30题数量允许0，额外要求每种至少1只时只有3组。
6. 第21题Fibonacci从1、1开始；第6题按逆序字符保留末尾0；第14/15题用long时限制项数不超过18。
7. 普通double小数运算存在表示误差；long、int都有上限，不应任意扩大输入范围后仍假设结果精确。

目录

学习起点：先认识 Java 程序 4

第 01 题 三个整数从大到小排列 7

第 02 题 分段函数 8

第 03 题 switch 四则运算 9

第 04 题 交错奇数求和 11

第 05 题 分数数列前30项和 12

第 06 题 位数与数字逆序 14

第 07 题 调用方法判断水仙花数 15

第 08 题 统计字符串中的数字字符 17

第 09 题 被2或3整除的数之和 18

第 10 题 最大的三位数约数 19

第 11 题 n个整数中的最大值 21

第 12 题 闭区间整数求和 22

第 13 题 四个数字组成不重复三位数 23

第 14 题 重复数字数列求和 25

第 15 题 原题第14题的示例（保留编号） 26

第 16 题 最大公约数和最小公倍数 27

第 17 题 m以内的水仙花数 29

第 18 题 阶乘方法与求和 30

第 19 题 输入0结束并求平均 32

第 20 题 数组原地逆序 33

第 21 题 数组保存Fibonacci前30项 34

第 22 题 20个数的最小值 36

第 23 题 方法返回课程平均分 37

第 24 题 冒泡排序升序 38

第 25 题 日期是当年第几天 40

第 26 题 有序数组二分查找 41

第 27 题 矩阵最大最小值及位置 43

第 28 题 数组顺序查找 44

第 29 题 四位回文数 46

第 30 题 百钱百鸡枚举 47

复习与参考 49

学习起点：先认识 Java 程序

推荐顺序：先运行最小程序，再读概念，再手写题目。每次只改一小处，编译后观察结果；报错时从第一条错误开始看。

最小可运行程序

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello Java");
    }
}
```

Main 是类名，必须与 Main.java 的文件名一致。main 是程序入口，花括号决定代码所属范围；println 表示输出一行，英文分号结束一条语句。

本地练习：安装 JDK 17，在 Main.java 所在目录运行 javac -encoding UTF-8 Main.java，再运行 java Main。javac 负责检查并编译，java 执行生成的字节码。编译失败时不会产生新的正确程序。

网站练习：选一道题，在编辑区输入代码，在标准输入区填数据，点击“编译并运行”。先观察编译状态，再读程序输出；“样例期望输出”仅用于对照。

1. 第0课：从一段能运行的代码开始

Java 源代码是写在文本文件里的指令。先安装 JDK 17（包含 javac 编译器与 java 运行器），再选择一个空文件夹。把下面代码保存为 Main.java，文件名与 public class 后面的 Main 必须完全一致。打开该文件夹的终端，输入 javac -encoding UTF-8 Main.java，成功时通常没有任何文字；这表示产生了 Main.class。再输入 java Main，屏幕显示 Hello Java。javac 后面是源文件名，java 后面是类名，不带 .java 或 .class。每一题都单独使用一个文件夹或覆盖同一个 Main.java，不要把30个 public class Main 放进同一文件。大括号配对表示代码范围；main 是本资料中所有程序的执行入口；分号表示一条语句结束。网页里的“编译并运行”会替你执行对应过程，练习时仍建议自己在电脑终端成功运行一次。

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello Java");
    }
}
```

2. 第1课：变量、类型与赋值

变量可以理解为带名字、只能存指定类型数据的格子。int age=20 声明一个整数变量，并把20放进去；age=21 是覆盖旧值，不是数学等式。int 常用于整数、下标、个数，范围约为负21亿到正21亿；long 范围更大，用于大整数及总和，整数字面量过大时写 10000000000L。double 用于小数与平均值；boolean 只有 true 和 false；char 用单引号表示一个字符，如 '7'；String 用双引号表示一段文本，如 "Java"。变量先声明再使用，局部变量在读取之前必须已被赋值。+、-、*、/ 分别做四则运算，% 求余数。两个整数做除法会舍弃小数，例如 7/2=3；7.0/2=3.5。sum+=x 等价于 sum=sum+x，i++ 等价于 i=i+1。double 采用二进制近似表示，0.1+0.2 可能出现很长小数，固定显示两位不等于内部变成精确十进制。

3. 第2课：如何给程序输入数据

使用 Scanner 前写 import java.util.Scanner;，在 main 中创建 Scanner sc=new Scanner(System.in);。sc.nextInt() 读下一个整数，nextLong() 读 long，nextDouble() 读小数。这些读取方法把空格和换行都当分隔符；输入 3 5 与分两行输入通常等价。sc.nextLine() 则读整行，能保留中间空格，适合第8题。若先 nextInt() 再 nextLine()，前者可能留下行尾换行符，后者会读到空行；这时要先额外 nextLine() 消耗剩余行尾。本资料每题的“标准输入”框就是送给 Scanner 的内容，不需要再输入提示文字。固定数组或固定求和题没有输入，输入框留空。题11、23、26、28先输入 n，再输入 n 个数据；题22恰好输入20个数，不要额外加n；题14、15统一按 n a 输入。数据少于程序要求时会等待或抛出异常，因此要先看题目的输入格式。

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int number = sc.nextInt();
        double score = sc.nextDouble();
        System.out.println(number);
        System.out.println(score);
    }
}
```

4. 第3课：条件分支决定做什么

if(条件){语句} 表示条件为真时才执行。if…else 表示二选一，if…else if…else 表示按顺序只选一个分支，适合分段函数。比较运算符有 >、>=、<、<=、==、!=；== 判断相等，= 是赋值。&& 表示同时满足，|| 表示至少满足一个，！ 表示取反。比如 x>=1 && x<=9 表示 x 在1到9之间；不能照数学写 1<=x<=9。&& 和 || 都会短路，例如先判断下标合法，再访问数组，能避免无效访问。switch 根据一个整数或字符值匹配 case，适合第3题的 +、-、*、/；每个分支一般用 break 结束，default 处理没有匹配的情况。分支题务必测边界：例如第2题的0和20、第25题的1900与2000、第29题的999和1000。

5. 第4课：for、while、break与return

知道重复次数时常用 for(初始化;继续条件;每轮更新){循环体}。例如 for(int i=1;i<=3;i++) 依次处理1、2、3。执行顺序是先初始化一次，再“判断条件→执行循环体→更新”，重复这个过程。不知道次数但知道结束条件时常用 while，例如不断去掉一个整数的末位，或读到0为止。累加器放在循环外并初始化为0，乘积累积初始化为1，计数器通常初始化为0。break 只结束它所在的最内层循环或 switch；return 结束整个当前方法，main 中 return 会结束主流程。嵌套循环中，外层每走一次，内层会完整运行一遍；第13题三个数字位置需要三重循环。检查循环的三个问题：第一次处理谁、最后一次处理谁、每轮是否真的向结束前进。循环条件不变或缺少更新会形成死循环，网页运行时间限制出现时应首先检查这一点。

6. 第5课：数组和二维数组

数组把同类型多个数放在一起。int[] a={8,6,5} 创建并赋初值；int[] a=new int[n] 创建长度为n的数组，整数元素初值为0。a.length 是长度，a[0] 是第一个元素，最后一个是 a[a.length-1]；长度为0的数组没有任何合法下标。遍历通常写 for(int i=0;i<a.length;i++)。数组的 length 没有括号，字符串的 length() 有括号，这是两种不同用法。交换两个元素要用临时变量：保存左值、覆盖左边、再恢复右边。第20题是翻转位置，第24题是按大小排序，二者不是同一操作。二维数组 int[][] a=new int[3][4] 可理解为3行、每行4个格子，a[i][j] 表示第i行第j列，i和j仍从0开始；向读者报告日常行号时可在输出处加1。最大值最小值应从第一个真实元素开始，不能一律从0开始。

7. 第6课：方法把一个任务单独写清楚

方法把能重复使用的一项计算命名。public static long fact(int n) 中，fact 是方法名，int n 是传入参数，long 是返回值类型；方法体通过 return result 把答案交给调用者。long x=fact(4) 会先执行 fact，再把返回值放到x。返回类型为 void 的方法只执行动作，不返回数值。本资料为便于入门把方法都放在 Main 类内，并使用 static，让静态的 main 能直接调用。局部变量只在自己的方法或大括号范围内存在，fact 的 result 与 main 的 sum 不是同一个格子。数组可以作为参数，例如 fun(double[] scores)，方法可以通过 scores.length 获得长度。第7题 is 返回 boolean，用于 if；第16题 gcd 与 lcm 各负责一种计算；第23题 fun 返回平均分；第26、28题返回查找下标。先明确“输入什么、返回什么”，再写方法体，比把所有计算挤在main里更容易检查。

```
public class Main {
    public static int add(int a, int b) {
        return a + b;
    }
    public static void main(String[] args) {
        int result = add(3, 5);
        System.out.println(result);
    }
}
```

8. 第7课：读懂题库中的五种常用思路

第一种是枚举：把所有可能依次尝试，用条件筛出答案，见水仙花数、三位数组合、百钱百鸡。第二种是逐步维护一个答案：每读到一个数就更新总和、个数、最大最小值，见第9、11、19、22、27题。第三种是递推：当前项由已经知道的项计算，见分数数列、重复数字、Fibonacci。第四种是位置操作：用下标交换、比较、缩小区间，见数组逆序、冒泡排序、二分查找。第五种是数字拆分：%10取得个位，/10去掉个位，见逆序、回文与水仙花数。先识别题目属于哪种思路，再确定保存哪些变量，最后写循环。不要一上来背整段代码：例如“求和”要有sum，“计数”要有count，“找位置”要有下标，“求平均”要同时有总和与个数。

9. 第8课：运行、核对和定位错误

每题先阅读题意与样例，再不看答案手算一次。复制完整代码到 Main.java，编译成功后再运行，把给定标准输入逐字输入。编译失败通常是语法或命名问题：少分号、大括号不配对、文件名不是Main.java、使用中文标点。程序运行后报错通常要看异常和行号：InputMismatchException 常见于把字母输入给 nextInt；NoSuchElementException 常见于输入不足；ArrayIndexOutOfBoundsException 表示下标越界。能运行但答案不对，则列出每轮变量值在纸上跟踪。编译成功并不证明算法正确，至少再试一个边界：全部负数、只有一个元素、目标不存在、输入0立即结束、闰年二月。网页“编译并运行”执行你当前编辑的代码，“参考答案”展示参考代码，题目中的样例输入和期望输出方便核对。网站在浏览器内编译运行；首次使用需要联网加载 Java 环境，遇到网络问题可下载 Main.java 在本地 JDK 运行。

10. 第9课：怎样从会复制变成会独立完成

建议分五次学习，每次集中完成一类任务。第一次做1、2、3、8、29题，练变量、输入、分支和字符；第二次做4、6、9—15、19题，练循环、求和、计数与数字构造；第三次做5、7、16—18题，练递推和方法；第四次做20—24、27题，练数组、二维数组和排序；第五次做25、26、28、30题，练边界、查找和综合枚举。每题按“手算样例→解释变量→逐行跟踪→关答案重写→改一个条件重做”的顺序练。判断掌握的标准是：能说清每个变量的作用，能解释循环为什么停止，能预测边界输出，能在改题后自行调整，而不仅是运行一次通过。附带练习不提供唯一答案，你可以直接在网站编辑区修改参考程序并运行验证。

如何判断自己学会了

合上答案，能说出输入、处理过程和输出；能自己写出主要代码；能解释循环每轮变量怎么变化；能修改样例并预测结果。只看懂代码不等于会写，至少独立重写一次。

第 01 题 三个整数从大到小排列

题目

从键盘输入三个整数 a、b、c，按从大到小的顺序输出。

基础思路

先把较大的数交换到 a，再把剩余两个数中较大的数交换到 b。

先掌握这些知识

int变量、Scanner输入、if比较、临时变量交换

一步一步理解

1. Scanner 把键盘输入读入三个 int 变量。输入的三个数可以放在一行，用空格分隔，也可以分行输入。

2. a < b 表示“a 比 b 小”。若成立，就应该让 b 的旧值成为 a 的新值，因此进行交换。

3. 交换必须借助 temp：先保存 a，再执行 a = b，最后执行 b = temp。直接连续写 a = b、b = a 会丢失原来的 a。

4. 第一次比较后 a 不小于 b，第二次比较后 a 也不小于 c，因此 a 是最大值。

5. 第三次只比较 b 和 c，让 b 不小于 c。此时整体满足 a >= b >= c，输出即可。

6. 例如 3、9、5：第一次变成 9、3、5，第二次不交换，第三次变成 9、5、3。相等的数字也能正常处理。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int a = sc.nextInt();
        int b = sc.nextInt();
        int c = sc.nextInt();
        int temp;
        if (a < b) { temp = a; a = b; b = temp; }
        if (a < c) { temp = a; a = c; c = temp; }
        if (b < c) { temp = b; b = c; c = temp; }
        System.out.println(a + " " + b + " " + c);
    }
}
```

样例输入

```
3 9 5
```

样例输出

```
9 5 3
```

容易出错的地方

1. 比较用 < 或 >，赋值用 =；它们的作用不同。
2. 字符串中的空格用于分隔结果，不能省略，否则 9、5、3 会连成 953。

自己动手

1. 把比较符号改为相反方向，实现从小到大排列。
2. 手算输入 -2、0、-2 时每次比较后的变量值。

第 02 题 分段函数

题目

输入实数 x，计算 f(x)：x <= 0 时为 $x^3 + 5$；0 < x <= 20 时为 $x^2 - x + 3$；x > 20 时为 7x + 2。输出保留两位小数。

基础思路

按区间顺序判断 x，只执行对应的一条计算公式。

先掌握这些知识

double、if / else if / else、边界条件、格式化输出

一步一步理解

1. x 可能带小数，所以使用 double 和 nextDouble()；y 也使用 double 保存函数值。
2. 先判断 x <= 0。这一分支成立时直接计算 x*x*x+5，后面的两个分支不会再执行。
3. 能进入 else if 就说明前一个条件为假，即 x > 0。因此第二个条件只写 x <= 20 就已完整表示 0 < x <= 20。
4. 前两个条件均不成立时，只剩 x > 20，最后的 else 使用 7*x+2。
5. 在 Java 中，^ 是按位异或，不是乘方。基础写法用 x*x 表示平方，用 x*x*x 表示立方。
6. %.2f 表示输出两位小数，%n 表示换行。Locale.US 固定使用小数点，便于与样例比较；它不影响公式本身。
7. 必须检查区间端点：x=0 属于第一段，结果 5；x=20 属于第二段，结果 383。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

Main.java（续）

```
        double x = sc.nextDouble();
        double y;
        if (x <= 0) {
            y = x * x * x + 5;
        } else if (x <= 20) {
            y = x * x - x + 3;
        } else {
            y = 7 * x + 2;
        }
        System.out.printf(java.util.Locale.US, "%.2f%n", y);
    }
}
```

样例输入

```
3
```

样例输出

```
9.00
```

容易出错的地方

1. 不能直接写 0 < x <= 20；Java 中应写 x > 0 && x <= 20，或采用本题的分支顺序。

2. double 可能存在微小误差，输出保留两位小数是显示要求，不代表内部计算变成了精确十进制。

自己动手

1. 手算 x=-2、0、20、21 分别进入哪个分支。

2. 把输出改为保留三位小数，并解释改变的是显示还是公式。

第 03 题 switch 四则运算

题目

输入一个简单四则表达式，例如 12-8，使用 Scanner 和 switch 输出表达式及计算结果。本答案允许一个二元运算、普通小数、操作数前的正负号和空格，不处理括号、科学计数法或连续多个运算。

基础思路

先找运算符，把左右两部分转成数，再用 switch 执行对应运算。

先掌握这些知识

字符串、char、substring、switch、double

一步一步理解

1. nextLine() 读取一整行；replace(" ", "") 去掉普通空格，因此输入 12 - 8 也可以。String 是字符串，char 是单个字符。

2. 用下标扫描表达式。下标从 1 开始，是为了跳过第一个操作数可能带有的正负号。例如 -12+8 中第一个减号不是二元运算符。

3. 找到运算符后记录 position 并 break，表示停止搜索。例如 12-8 的 position 是 2，下标从 0 开始。

4. substring(0, position) 取得左边字符，substring(position+1) 取得右边字符；Double.parseDouble 把这些字符转换为可计算的数。

5. switch 根据 op 的字符值选择 case。每个 case 后的 break 防止继续执行后面的分支；除法前额外排除除数为 0。

6. a 和 b 都是 double，因此 7/2 会得到 3.5。如果把它们都声明为 int，整数除法只会得到 3。

7. 最后把数值拼接成“表达式=结果”。若结果是 long 范围内的整数，就去掉 .0，使示例严格输出 12-8=4。

8. 本题重点是 switch。它不是通用表达式解析器；请按题目约定输入一个完整、合法的二元表达式，例如 -12--8。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String expression = sc.nextLine().replace(" ", "");
        int position = -1;
        for (int i = 1; i < expression.length(); i++) {
            char ch = expression.charAt(i);
            if (ch == '+' || ch == '-' || ch == '*' || ch == '/') {
                position = i;
                break;
            }
        }
        if (position == -1) {
            System.out.println("表达式格式错误");
            return;
        }
        double a = Double.parseDouble(expression.substring(0, position));
        double b = Double.parseDouble(expression.substring(position + 1));
        char op = expression.charAt(position);
        double result = 0;
        switch (op) {
            case '+': result = a + b; break;
            case '-': result = a - b; break;
            case '*': result = a * b; break;
            case '/':
                if (b == 0) {
                    System.out.println("除数不能为0");
                    return;
                }
                result = a / b;
                break;
            default:
                System.out.println("不支持的运算符");
                return;
        }
        String answer;
```

Main.java（续）

```
        if (result >= Long.MIN_VALUE && result < Long.MAX_VALUE
                && result == (long) result) {
            answer = String.valueOf((long) result);
        } else {
            answer = String.valueOf(result);
        }
        System.out.println(expression + "=" + answer);
    }
}
```

样例输入

```
12-8
```

样例输出

```
12-8=4
```

容易出错的地方

1. case 后通常需要 break；漏掉会继续执行后续 case。
2. 输入乘号应为 *，除号应为 /，不是中文的 × 和 ÷。
3. 0.1+0.2 可能显示 0.30000000000000004，这是 double 的表示误差；需要固定小数位时可另用 printf。

自己动手

1. 把最后的结果改为始终保留两位小数。
2. 在 switch 中增加求余 %，并说明它与除法的区别。

第 04 题 交错奇数求和

题目

计算 1-3+5-7+…-99+101 的值。

基础思路

用 i 依次取所有奇数，每加完一项便把符号乘以 -1。

先掌握这些知识

for循环、累加器、符号切换

一步一步理解

1. sum 是累加器，起始为 0；sign 保存当前项的符号，起始为 1 表示正号。
2. for 中 i 从 1 开始，每次增加 2，所以依次得到 1、3、5……101，正好覆盖题目中的奇数。
3. 先执行 sum = sum + sign*i，把带符号的当前项加入总和。

4. 再执行 sign = -sign，使 1 和 -1 交替出现。前几步分别是 0+1、1-3、-2+5。

5. i<=101 包含最后的 +101。循环共执行 51 次，最后输出 51。

6. 可以独立验算：(1-3)+(5-7)+…+(97-99)+101，共 25 个 -2 再加 101，结果是 51。

完整参考代码 · Main.java

```
public class Main {
    public static void main(String[] args) {
        int sum = 0;
        int sign = 1;
        for (int i = 1; i <= 101; i += 2) {
            sum = sum + sign * i;
            sign = -sign;
        }
        System.out.println(sum);
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
51
```

容易出错的地方

1. i++ 会遍历所有整数，不能代替 i+=2。

2. sign 的取反应放在累加之后，否则第一项会成为 -1。

自己动手

1. 改为计算 1-3+5-…-199+201。

2. 尝试用一个布尔变量决定当前项相加还是相减。

第 05 题 分数数列前30项和

题目

求 2/1、3/2、5/3、8/5、13/8、21/13……的前 30 项之和，结果保留六位小数。

基础思路

分子与分母都来自相邻的 Fibonacci 数；新分子等于旧分子加旧分母，新分母等于旧分子。

先掌握这些知识

递推、double除法、临时变量、累加

一步一步理解

1. numerator 表示分子，denominator 表示分母。初值 2 和 1 对应第一项 2/1。

2. 观察相邻项：2/1 之后是 3/2，其中 3=2+1，2 是旧分子；再下一项 5/3 也符合这一规律。

3. 每轮先求 numerator/denominator 并累加，确保第一轮加入的是 2/1，而不是提前更新后的 3/2。

4. next 暂存“旧分子+旧分母”，然后把旧分子交给分母，最后把 next 交给分子，完成一次递推。

5. 循环从 1 到 30，共累加 30 项。最后一次更新虽不再使用，但不影响已经得到的总和。

6. 必须进行实数除法。这里把分子分母定义成 double，因此 3/2 会按 1.5 计算；若都用 int，会错误地得到 1。

7. 结果以六位小数显示为 48.840601。浮点运算只保存近似值，学习时应比较指定小数位，而不是要求所有位都精确。

完整参考代码 · Main.java

```
public class Main {
    public static void main(String[] args) {
        double numerator = 2;
        double denominator = 1;
        double sum = 0;
        for (int i = 1; i <= 30; i++) {
            sum = sum + numerator / denominator;
            double next = numerator + denominator;
            denominator = numerator;
            numerator = next;
        }
        System.out.printf(java.util.Locale.US, "%.6f%n", sum);
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
48.840601
```

容易出错的地方

1. 不能先写 numerator=numerator+denominator，再写 denominator=numerator，否则分母取到的是新分子。

2. 初值是 2/1，不是 Fibonacci 数列最常见的 1/1。

自己动手

1. 把项数 30 改为由键盘输入的 n。

2. 只算前三项，手算结果 2+1.5+5.0/3，并与程序核对。

第 06 题 位数与数字逆序

题目

输入一个正整数 k，输出它的位数，并按逆序输出各位数字。本答案用 long 保存 k；逆序按字符显示，因此 1200 显示为 0021。第一行是位数，第二行是逆序结果。

基础思路

反复用 %10 取出个位，用 /10 去掉个位，同时计数并拼接。

先掌握这些知识

long、while、整数除法、取余、字符串拼接

一步一步理解

1. long 能保存比 int 更大的整数，但仍有范围限制；本题要求输入正整数，所以先排除 0 和负数。

2. 把 k 复制到 temp。循环会不断缩短 temp，原来的 k 则完整保留，便于理解和后续扩展。

3. temp%10 取得个位，例如 123%10=3。把它拼接到 reverse，第一次 reverse 从空字符串变为 "3"。

4. temp/10 使用整数除法去掉个位，例如 123/10=12。第二轮取得 2，第三轮取得 1，最后 temp 变成 0，循环结束。

5. 每取出一位就执行 count++，因此循环次数正好是十进制位数。输入 123 会循环三次。

6. 逆序结果用 String 保存，而不是重新组成整数。因此末尾的 0 也会保留：1200 的四位按相反顺序为 0021。

7. 若题目改成“输出逆序后的数值”，也可以用 reverseNumber=reverseNumber*10+digit；但那样 0021 会自然显示为 21，并且还要考虑数值溢出。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        long k = sc.nextLong();
        if (k <= 0) {
            System.out.println("请输入正整数");
            return;
        }
        long temp = k;
        int count = 0;
        String reverse = "";
        while (temp > 0) {
            reverse = reverse + temp % 10;
            count++;
            temp = temp / 10;
        }
        System.out.println(count);
```

Main.java（续）

```
        System.out.println(reverse);
    }
}
```

样例输入

```
123
```

样例输出

```
3
321
```

容易出错的地方

1. % 是取余，/ 对整数执行整除，两者不能混淆。

2. 不能在循环中一直使用不变的 k 作为条件，否则可能形成死循环。

自己动手

1. 把程序改为允许输入 0，并规定 0 的位数为 1。

2. 额外计算各位数字之和，例如 123 的各位和为 6。

第 07 题 调用方法判断水仙花数

题目

输出所有三位水仙花数，并定义、调用 is(number) 判断一个数的各位数字立方和是否等于自身。

基础思路

主方法枚举 100 到 999；is 方法只负责检查一个数是否符合定义。

先掌握这些知识

方法定义、boolean返回值、数字拆分、枚举

一步一步理解

1. public static boolean is(int number) 定义一个方法：传入一个 int，返回 true 或 false。static 让 main 可以直接调用它。

2. 先检查三位数范围。根据题目定义，一位数即使等于自身的立方，也不算这里要找的三位水仙花数。

3. number/100 取百位；number/10%10 先去掉个位再取个位，得到十位；number%10 取得原来的个位。

4. 分别用 a*a*a 等计算立方，再用 == 比较立方和与原数。比较表达式本身就是 boolean，可以直接 return。

5. main 用 for 依次尝试每个三位数。if(is(i)) 的意思是“如果 is 方法认定这个 i 符合要求，就输出它”。

6. 例如 153 被拆成 1、5、3，立方和 1+125+27=153，返回 true。154 的立方和不等于 154，返回 false。

7. first 仅控制输出空格，避免行首和行尾多余空格，与判断算法无关。最终得到四个数：153、370、371、407。

完整参考代码 · Main.java

```
public class Main {
    public static boolean is(int number) {
        if (number < 100 || number > 999) return false;
        int a = number / 100;
        int b = number / 10 % 10;
        int c = number % 10;
        return a * a * a + b * b * b + c * c * c == number;
    }
    public static void main(String[] args) {
        boolean first = true;
        for (int i = 100; i <= 999; i++) {
            if (is(i)) {
                if (!first) System.out.print(" ");
                System.out.print(i);
                first = false;
            }
        }
        System.out.println();
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
153 370 371 407
```

容易出错的地方

1. 方法名按题目使用 is，不要把它误写为关键字 if。

2. 相等判断用 ==；a*a*a 中的 * 才是乘法。

自己动手

1. 让用户输入一个数，只判断该数是否为三位水仙花数。

2. 统计水仙花数的个数，并在最后多输出一行。

第 08 题 统计字符串中的数字字符

题目

输入一整行字符串，统计其中数字字符 0 到 9 的个数。空格也属于这行字符串的一部分。

基础思路

从左到右检查每个字符，落在字符 0 到 9 范围内就把计数加一。

先掌握这些知识

String、nextLine、charAt、字符范围、计数

一步一步理解

1. nextLine() 读取完整一行，能保留中间的空格。next() 只读取一个单词，会漏掉空格后面的内容。

2. text.length() 给出字符串长度。字符串下标从 0 开始，所以最后一个有效下标是 length()-1。

3. charAt(i) 取得下标为 i 的单个字符，并用 char 变量 ch 保存。

4. '0' 和 '9' 是字符常量，它们带单引号；0 和 9 是整数，含义不同。普通数字字符的编码连续，因此可用范围比较。

5. && 表示两个条件同时成立：ch 不小于字符 0，而且不大于字符 9，才算一个数字字符。

6. 每找到一个数字字符，count++ 就增加 1。样例中的 2026 有四个数字，95 有两个，总共六个。

7. 这里按常见入门题统计半角数字 0—9；全角数字 １ ２ ３ 和其他 Unicode 数字不计入。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String text = sc.nextLine();
        int count = 0;
        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);
            if (ch >= '0' && ch <= '9') count++;
        }
        System.out.println(count);
    }
}
```

样例输入

```
Java 2026, score=95
```

样例输出

```
6
```

容易出错的地方

1. 循环条件应为 i<text.length()，不能写 <=，否则最后会访问越界。

2. 不能用 nextInt() 读取混有字母的整行字符串。

自己动手

1. 再统计英文字母的数量。

2. 分别统计十个数字出现的次数，尝试使用长度为 10 的计数数组。

第 09 题 被2或3整除的数之和

题目

输入正整数 n，求 1 到 n 中所有能够被 2 或者被 3 整除的数之和。

基础思路

枚举每个整数，只要满足两个整除条件之一，就累加一次。

先掌握这些知识

取余判断整除、逻辑或、for、long累加

一步一步理解

1. i%2==0 表示 i 除以 2 没有余数，也就是能被 2 整除；对 3 同理。

2. || 是逻辑“或”，两个条件至少有一个为真，整体就为真。6 同时满足两个条件，整体仍只执行一次累加。

3. sum 起始为 0。用 long 是因为多个整数相加可能超出 int 的范围，循环变量也用 long 以避免 int 终值递增溢出。

4. for 从 1 到 n，条件包含等号，因此 n 自己若符合要求也会被计入。

5. 当 n=10，符合条件的是 2、3、4、6、8、9、10，总和 42。

6. 不要分别开两个循环把所有 2 的倍数和所有 3 的倍数直接相加，这样会把 6 的倍数重复统计。

7. 这种枚举法需要检查 n 次，称为 O(n)；对课堂规模输入足够直观，极大的 n 可另学等差数列与容斥公式。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if (n <= 0) {
            System.out.println("请输入正整数");
            return;
        }
```

Main.java（续）

```
        long sum = 0;
        for (long i = 1; i <= n; i++) {
            if (i % 2 == 0 || i % 3 == 0) sum += i;
        }
        System.out.println(sum);
    }
}
```

样例输入

```
10
```

样例输出

```
42
```

容易出错的地方

1. 题目是“或者”，应使用 ||；&& 会变成同时能被 2 和 3 整除。
2. 累加器应在循环外初始化，不能每一轮重新设为 0。

自己动手

1. 改为求同时能被 2 和 3 整除的数之和。
2. 除了和，再输出符合要求的数的个数。

第 10 题 最大的三位数约数

题目

用户输入 n，要求 n>10000；求 n 的约数中最大的三位数。如果不存在三位数约数，明确输出“不存在三位数约数”。

基础思路

从 999 倒着检查到 100，找到第一个能整除 n 的数就结束。

先掌握这些知识

约数、倒序枚举、return、long

一步一步理解

1. 如果 n%i==0，那么 i 是 n 的约数。题目限制约数为三位数，因此只检查 100—999。
2. 从大到小检查的好处是：第一个满足条件的数一定最大，不必再比较或保存所有约数。
3. 循环变量 i 从 999 开始，每次减 1，直到 100 为止。这个范围只有 900 个候选数，与 n 的大小无关。
4. 找到约数后输出并 return。main 中的 return 会直接结束程序，不会再打印“找不到”的信息。

5. 样例 12345=15×823，程序从 999 向下检查，首先找到 823，所以输出 823。

6. n>10000 并不能保证存在三位数约数，例如 10007 是质数。因此原题“执行后显示一个三位数”需要补充不存在时的处理。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        long n = sc.nextLong();
        if (n <= 10000) {
            System.out.println("n必须大于10000");
            return;
        }
        for (int i = 999; i >= 100; i--) {
            if (n % i == 0) {
                System.out.println(i);
                return;
            }
        }
        System.out.println("不存在三位数约数");
    }
}
```

样例输入

```
12345
```

样例输出

```
823
```

容易出错的地方

1. 从 100 向上找到就立即结束，只会得到最小的三位数约数。

2. 条件应为 n%i==0，不是 i%n==0。

自己动手

1. 修改程序，输出全部三位数约数。

2. 把枚举范围改成 10—99，寻找最大两位数约数。

第 11 题 n个整数中的最大值

题目

先输入整数个数 n，再输入 n 个整数，输出其中最大值，格式为 max=数值。n 必须大于 0。

基础思路

把第一个实际输入值当成当前最大值，再依次比较其余 n-1 个数。

先掌握这些知识

动态输入数量、最大值维护、负数边界

一步一步理解

1. 第一个输入 n 表示后面有多少个数据，不属于要比较的数据本身。
2. 没有数据就不存在最大值，所以先检查 n>0。接着读取第一个数并赋给 max。
3. max 必须来自实际数据，不能随意初始化为 0。例如所有数据都是负数时，0 并没有出现，却会被错误当成最大值。
4. 已经读过一个数，所以循环从 i=1 开始，到 i<n 为止，正好再读 n-1 个数。
5. 每次只需要保存当前读到的 value；若 value>max，就用它更新 max，否则维持原值。
6. 循环结束后，max 已经和每个输入数比较过，因此是整组数据的最大值；不必使用数组。
7. 当 n=1 时循环一次也不执行，第一个数直接就是最大值，这也是应检查的边界情况。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if (n <= 0) {
            System.out.println("n必须大于0");
            return;
        }
        int max = sc.nextInt();
        for (int i = 1; i < n; i++) {
            int value = sc.nextInt();
            if (value > max) max = value;
        }
        System.out.println("max=" + max);
    }
}
```

样例输入

```
5
3 9 -2 7 5
```

样例输出

```
max=9
```

容易出错的地方

1. 不要把 n 也拿去和后面的数据比较。
2. 若循环又读取 n 次，总共就会要求 n+1 个数据，导致输入不足。

自己动手

1. 同时求最小值 min。
2. 记录最大值在输入序列中第一次出现的位置，从 1 开始计数。

第 12 题 闭区间整数求和

题目

输入两个整数 m、n，要求 m<=n；求从 m 到 n 的所有整数之和，包含两个端点，输出 S=结果。

基础思路

从 m 开始逐个累加到 n，使用 long 保存总和。

先掌握这些知识

区间枚举、累加、long

一步一步理解

1. 题目中的 m~n 表示闭区间，m 和 n 都必须参加求和，所以循环从 m 开始并使用 i<=n。
2. sum 的初值是 0，每轮 sum+=i 等价于 sum=sum+i。
3. 输入 3 和 5 时依次累加 3、4、5，因此 S=12；输入 m=n 时只累加一次。
4. m 和 n 可以为负数，例如 -2 到 2 的和为 -2-1+0+1+2=0，循环无需特殊修改。
5. 循环变量和总和都用 long。这样即便 n 是 int 的最大值，最后一次 i++ 也不会使 int 溢出后继续循环。
6. 枚举法容易理解，但跨度很大的区间会运行很久。熟悉基础后可用等差数列公式 (m+n)×项数÷2，并注意先转 long 再做加法。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int m = sc.nextInt();
        int n = sc.nextInt();
        if (m > n) {
            System.out.println("m不能大于n");
```

Main.java（续）

```
            return;
        }
        long sum = 0;
        for (long i = m; i <= n; i++) sum += i;
        System.out.println("S=" + sum);
    }
}
```

样例输入

```
3 5
```

样例输出

```
S=12
```

容易出错的地方

1. 若条件误写成 i<n，会漏掉 n。
2. 用公式时不能先以 int 计算 m+n，再把已经溢出的值赋给 long。

自己动手

1. 改用等差数列公式，并与循环法比较结果。
2. 只累加区间中的奇数。

第 13 题 四个数字组成不重复三位数

题目

用 1、2、3、4 组成互不相同且每个数内部无重复数字的三位数，输出全部三位数和总个数。

基础思路

百、十、个位分别枚举 1—4，只有三个数字两两不同才输出。

先掌握这些知识

三重循环、逻辑与、排列枚举

一步一步理解

1. 用 a、b、c 分别代表百位、十位、个位，每个位置都有 1—4 四种候选。
2. 三重循环会检查 4×4×4=64 种组合。最内层的一轮对应一组三位数字，例如 a=1、b=2、c=3。
3. “无重复数字”要求 a!=b、a!=c、b!=c 三个条件同时成立，因此使用 && 连接。
4. 满足条件后，用 100*a+10*b+c 计算三位数；这里每一项都是数值运算，不是把字符随意拼接。
5. count 既统计已找到的数量，也帮助输出空格。第一个数前不加空格，后续每个数前加一个空格。

6. 每个数的三个位置都唯一确定，因此枚举不会重复产生同一个三位数。循环顺序还使输出自然按从小到大排列。

7. 独立计数：百位有 4 种，十位剩 3 种，个位剩 2 种，共 4×3×2=24，与程序输出相符。

完整参考代码 · Main.java

```
public class Main {
    public static void main(String[] args) {
        int count = 0;
        for (int a = 1; a <= 4; a++) {
            for (int b = 1; b <= 4; b++) {
                for (int c = 1; c <= 4; c++) {
                    if (a != b && a != c && b != c) {
                        if (count > 0) System.out.print(" ");
                        System.out.print(100 * a + 10 * b + c);
                        count++;
                    }
                }
            }
        }
        System.out.println();
        System.out.println("总数=" + count);
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
123 124 132 134 142 143 213 214 231 234 241 243 312 314 321 324 341 342 412 413 421 423 431 432
总数=24
```

容易出错的地方

1. 只检查 a!=b 和 b!=c 会漏掉 a=c 的重复情况。

2. 题目不允许 0，因此不需要额外处理三位数的前导 0。

自己动手

1. 改用数字 0、1、2、3，并排除百位为 0 的情况。

2. 只输出其中的偶数，并统计个数。

第 14 题 重复数字数列求和

题目

计算 Sn=a+aa+aaa+…（共 n 项）。按原题示例统一输入顺序为 n a。为使用基础 long 且不溢出，本答案约定 1<=n<=18、0<=a<=9。

基础思路

每一项由上一项乘 10 再加 a 得到，随后加入总和。

先掌握这些知识

十进制构造、long、递推累加

一步一步理解

1. 先输入项数 n，再输入一个数字 a。样例 3 2 的意思是计算三项 2、22、222，而不是计算两项数字 3。
2. term 保存当前项，初值为 0。十进制整数乘 10 相当于在末尾空出一位，再加 a 就把 a 填入这一位。
3. a=2 时：第一轮 term=0*10+2=2；第二轮 term=2*10+2=22；第三轮得到 222。
4. 每生成一项就加入 sum。三轮后 sum 依次是 2、24、246，循环结束时已经包含全部 n 项。
5. term 和 sum 都用 long，因为类似 2222222222 已经超过 int 上限。
6. long 也不是无限大。为覆盖 a=9 的最坏情况且保持基础写法，这份答案限制 n<=18；更长的项数应另学 BigInteger。
7. 原题第 15 条实际上是本题的说明与样例，并非另一种数列；为对应全部编号，本资料仍单独保留第 15 题。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int a = sc.nextInt();
        if (n < 1 || n > 18 || a < 0 || a > 9) {
            System.out.println("要求1<=n<=18且0<=a<=9");
            return;
        }
        long term = 0;
        long sum = 0;
        for (int i = 1; i <= n; i++) {
            term = term * 10 + a;
            sum = sum + term;
        }
        System.out.println(sum);
    }
}
```

样例输入

```
3 2
```

样例输出

```
246
```

容易出错的地方

1. 表达式 aa 不是 Java 变量 a 的平方，也不是 a+a，应通过 term*10+a 构造。
2. sum 与 term 是不同变量：一个保存总和，一个保存当前项。

自己动手

1. 输入 4 3，手动跟踪每轮 term 和 sum。
2. 增加输出每一个加数，让结果显示为 2+22+222=246。

第 15 题 原题第14题的示例（保留编号）

题目

原文第 15 条是第 14 题续写的示例：S5=2+22+222+2222+22222。继续采用输入 n a；原文样例 3 2 的结果为 246。这里保留独立编号方便对照，算法与第 14 题相同。

基础思路

仍用 term=term*10+a 逐项生成，以 n 控制项数；数字 2 由输入提供。

先掌握这些知识

阅读题意、重复数字构造、循环跟踪

一步一步理解

1. 这一条不是新算法，而是对第 14 条的举例。S5 的下标 5 表示五项，不是每一项都只有五位。
2. 按 n a 的顺序输入 5 2，让 n=5、a=2；若按照原文另一组输入 3 2，则只计算前三项。
3. 初始 term=0、sum=0。第 1 轮产生 2，sum=2；第 2 轮产生 22，sum=24。
4. 第 3 轮产生 222，sum=246；第 4 轮产生 2222，sum=2468；第 5 轮产生 22222，sum=24690。
5. 代码不把 a 写死为 2，因此也能完成第 14 条一般化的要求；例如输入 2 3，得到 3+33=36。
6. 同样采用 long 并限制 1<=n<=18、0<=a<=9，避免在基础练习中悄悄发生溢出。
7. 学习时建议先手算每一轮的两列：当前项 term、累计和 sum，再运行程序核对，而不是仅记住最终数字。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int a = sc.nextInt();
        if (n < 1 || n > 18 || a < 0 || a > 9) {
            System.out.println("要求1<=n<=18且0<=a<=9");
            return;
        }
        long term = 0;
        long sum = 0;
        for (int i = 1; i <= n; i++) {
            term = term * 10 + a;
            sum = sum + term;
        }
        System.out.println(sum);
    }
}
```

样例输入

```
5 2
```

样例输出

```
24690
```

容易出错的地方

1. 原文同时出现“n=5”的讲解与“3 2”的输入样例，二者属于不同示例，输出应分别是 24690 和 246。
2. 不要因为示例使用 2 就把通用输入顺序改成 a n。

自己动手

1. 保持 a=2，依次输入 n=1、2、3、4、5，观察累计和。
2. 输入 2 3，解释为什么结果是 36 而不是 26。

第 16 题 最大公约数和最小公倍数

题目

输入两个整数，由主方法分别调用两个方法求最大公约数与最小公倍数。第一行输出最大公约数，第二行输出最小公倍数。支持负数和单个 0；两个数同时为 0 时提示无定义。

基础思路

用辗转相除法求 gcd，再用 |a/gcd*b| 求 lcm，先除后乘降低溢出风险。

先掌握这些知识

方法调用、辗转相除法、Math.abs、long

一步一步理解

1. 两个输入按 int 范围读取，再存入 long。这样负数取绝对值和两数乘积都能在 long 范围内安全处理；本方法不承诺任意 long 输入均不溢出。

2. 最大公约数通常取非负，因此 gcd 开头用 Math.abs 把两个数转为绝对值。

3. 辗转相除的依据是 gcd(a,b)=gcd(b,a%b)。例如 gcd(18,12) 可以变成 gcd(12,6)，再变成 gcd(6,0)。

4. 循环中 remainder 先保存余数，再令 a=b、b=remainder；当 b 为 0 时，a 就是最大公约数。

5. 对于不为 0 的两数，最小公倍数等于两数乘积的绝对值除以最大公约数。代码先执行 a/gcd，再乘 b，可减小中间结果。

6. lcm 方法遇到任一输入为 0，就按常见程序约定返回 0。gcd(0,b)=|b|；两个数都是 0 时，本答案选择明确提示无定义。

7. main 只负责输入、处理特殊情况、调用并输出，实际计算由 gcd 和 lcm 完成，这就符合“用主函数调用两个函数”的要求。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static long gcd(long a, long b) {
        a = Math.abs(a);
        b = Math.abs(b);
        while (b != 0) {
            long remainder = a % b;
            a = b;
            b = remainder;
        }
        return a;
    }
    public static long lcm(long a, long b) {
        if (a == 0 || b == 0) return 0;
        return Math.abs(a / gcd(a, b) * b);
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        long a = sc.nextInt();
        long b = sc.nextInt();
        if (a == 0 && b == 0) {
            System.out.println("两个0的最大公约数无定义");
            return;
        }
        System.out.println(gcd(a, b));
        System.out.println(lcm(a, b));
    }
}
```

样例输入

```
12 18
```

样例输出

```
6
36
```

容易出错的地方

1. 每次计算余数必须在覆盖 a 和 b 之前完成。
2. 直接写 a*b/gcd 对 int 容易先溢出，即使最后结果较小也不能补救。

自己动手

1. 手算 gcd(48,18) 每一轮 a、b 的变化。
2. 用最基础的从小到大枚举公约数方法求 gcd，再比较运行步骤。

第 17 题 m以内的水仙花数

题目

输入 m（0<=m<=1000），输出不超过 m 的所有三位水仙花数。没有符合条件的数时输出“无”。

基础思路

从 100 枚举到 m，同时限制不超过 999，复用三位数拆分判断方法。

先掌握这些知识

方法复用、枚举上限、三位数范围

一步一步理解

1. 本题与第 7 题的定义相同，区别在于输出范围由用户输入的 m 决定。
2. 三位数最小是 100，所以从 100 开始枚举。即便 m 小于 100，循环也自然一次不执行。
3. 循环条件同时要求 i<=m 和 i<=999。因此 m=1000 时，1000 自身不会被当作三位数。
4. is 方法把候选数拆成百位、十位和个位，比较三个立方之和与原数。本方法只由三位数枚举循环调用。
5. found 初始为 false；只要找到一个合格数，就将它改成 true，也利用它控制数字之间的空格。
6. 循环结束后若 found 仍为 false，说明根本没有输出过数字，于是打印“无”，使结果可读。
7. m=370 时包含 370，因此输出 153 370；把条件写成 i<m 则会漏掉恰好等于上限的水仙花数。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static boolean is(int number) {
        int a = number / 100;
        int b = number / 10 % 10;
        int c = number % 10;
        return a * a * a + b * b * b + c * c * c == number;
```

Main.java（续）

```
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int m = sc.nextInt();
        if (m < 0 || m > 1000) {
            System.out.println("要求0<=m<=1000");
            return;
        }
        boolean found = false;
        for (int i = 100; i <= m && i <= 999; i++) {
            if (is(i)) {
                if (found) System.out.print(" ");
                System.out.print(i);
                found = true;
            }
        }
        if (!found) System.out.print("无");
        System.out.println();
    }
}
```

样例输入

```
1000
```

样例输出

```
153 370 371 407
```

容易出错的地方

1. 题目中的水仙花数只指三位数，不应把 1 之类的一位数列入。

2. m 的上限是 1000，但被检查的数仍应限制为三位。

自己动手

1. 改成输入下限 l 和上限 r，输出区间内的水仙花数。

2. 让程序同时输出找到的个数。

第 18 题 阶乘方法与求和

题目

编写 fact(n) 计算 n!，调用该方法计算并输出 4!+7!+8!。

基础思路

fact 使用循环从 1 乘到 n，主方法调用三次并相加。

先掌握这些知识

方法参数、返回值、阶乘、乘积累积

一步一步理解

1. n! 表示从 1 到 n 的所有整数相乘，例如 4!=1×2×3×4=24。

2. fact 的参数 n 决定乘到哪里；返回类型是 long，意味着调用者会得到一个 long 结果。

3. 乘积变量 result 必须初始化为 1。若初始化为 0，后面无论乘什么都会一直是 0。

4. 每次调用 fact 都会创建自己的一份局部 result 和 i；fact(4)、fact(7)、fact(8) 互不影响。

5. 主方法把三个返回值相加：24+5040+40320=45384。这里必须有方法调用，不能只把最终常数写入代码。

6. 对于 n=0，循环不执行并返回 1，恰好符合 0!=1。此基础方法的使用前提是 n 为非负整数。

7. long 最多精确保存到 20!；21! 已超出范围。当前只计算到 8!，不存在溢出问题，扩展时必须重新考虑数值类型。

完整参考代码 · Main.java

```
public class Main {
    public static long fact(int n) {
        long result = 1;
        for (int i = 1; i <= n; i++) result *= i;
        return result;
    }
    public static void main(String[] args) {
        long sum = fact(4) + fact(7) + fact(8);
        System.out.println(sum);
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
45384
```

容易出错的地方

1. Java 中 ！是逻辑非，不能直接写 4! 来求阶乘。

2. fact 的 return 应放在循环之后，否则第一次循环就提前结束。

自己动手

1. 输入 n（0<=n<=20）并输出 n!。

2. 调用 fact 求 1!+2!+…+10!，不要把 fact 改写成只计算和的方法。

第 19 题 输入0结束并求平均

题目

输入一组数，读到 0 时结束；0 不计入数据。输出个数和平均值，平均值保留一位小数。若第一项就是 0，约定输出 0 0.0，避免除以 0。

基础思路

读入后先判断是否为结束标记，再计数和累加，最后计算平均值。

先掌握这些知识

哨兵值、while、break、计数与求和、格式化

一步一步理解

1. 0 是“哨兵值”，只用来表示输入结束，不是要统计的普通数据。题目因此不能把一个真正的 0 分数放在数据中间。

2. hasNextDouble() 检查是否还有可读取的数，既支持题目规定的 0，也允许输入流自然结束时收尾。

3. 每轮先读取 value，再检查 value==0；如果成立就 break，跳出循环，因此 0 不会增加 count，也不会参与 sum。

4. 没有遇到 0 时，count++ 记录一个数据，sum+=value 累加它的值。使用 double 可以接收小数和负数。

5. 平均值等于总和除以个数。sum 是 double，所以除法保留小数，不会执行整数截断。

6. count 为 0 时数学平均值未定义；这里为了让程序正常输出，显式约定显示 0.0，而不是执行 0/0。

7. %d 对应整数个数，%.1f 对应一位小数平均值，两个格式之间的空格决定输出为“10 27.5”。

8. 样例十个数的和为 275，个数为 10，因此平均值 27.5；0 后面的数字不会再被统计。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int count = 0;
        double sum = 0;
        while (sc.hasNextDouble()) {
            double value = sc.nextDouble();
            if (value == 0) break;
            count++;
            sum += value;
        }
        double average = 0;
        if (count > 0) average = sum / count;
        System.out.printf(java.util.Locale.US, "%d %.1f%n", count, average);
    }
}
```

样例输入

```
5 10 15 20 25 30 35 40 45 50 0
```

样例输出

```
10 27.5
```

容易出错的地方

1. 先 count++ 再判断 0，会把结束标记错误统计进去。
2. 不应默认 count 一定大于 0。

自己动手

1. 增加最小值与最大值统计。
2. 把结束标记改为 -1，并思考这样会限制哪些合法数据。

第 20 题 数组原地逆序

题目

把数组 {8,6,5,4,1} 的值按逆序重新存放，再输出结果。注意要求改变数组内部顺序。

基础思路

交换首尾，再交换次首与次尾，只处理数组前半部分。

先掌握这些知识

数组、下标、对称位置交换

一步一步理解

1. int[] a 表示一个整型数组；五个元素的下标依次是 0、1、2、3、4。a.length 是元素个数 5。
2. 下标 i 的对称位置是 a.length-1-i。比如 i=0 对应 4，i=1 对应 3。
3. 通过 temp 交换两个位置的值，第一次交换后数组变为 {1,6,5,4,8}，第二次变为 {1,4,5,6,8}。
4. 循环只运行 a.length/2 次。长度 5 的整数除法结果为 2，中间的元素 5 不必移动。
5. 如果对整个数组都交换一次，后半段会把前半段的交换撤销，最后反而恢复原顺序。
6. 第二个循环从前往后输出已修改的数组。只倒着打印而不改变数组，不能满足“重新存放”的要求。
7. 本题示例刚好逆序后呈升序，但算法只负责翻转，不负责排序；例如 {2,9,1} 翻转是 {1,9,2}。

完整参考代码 · Main.java

```
public class Main {
    public static void main(String[] args) {
        int[] a = {8, 6, 5, 4, 1};
        for (int i = 0; i < a.length / 2; i++) {
```

Main.java（续）

```
            int j = a.length - 1 - i;
            int temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
        for (int i = 0; i < a.length; i++) {
            if (i > 0) System.out.print(" ");
            System.out.print(a[i]);
        }
        System.out.println();
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
1 4 5 6 8
```

容易出错的地方

1. 最后一个元素的下标是 length-1，不是 length。

2. 逆序与排序是不同操作。

自己动手

1. 把数组改成 {2,9,1}，解释输出为何不是升序。

2. 让用户输入 n 个整数，再原地翻转。

第 21 题 数组保存Fibonacci前30项

题目

用数组保存并输出 Fibonacci 数列前 30 项。本答案采用第 1 项为 1、第 2 项为 1 的约定。

基础思路

先保存两个初值，从下标 2 开始，让当前项等于前两项之和。

先掌握这些知识

数组递推、初值、下标与项号

一步一步理解

1. Fibonacci 数列从第三项开始，每项都等于前两项之和。本题未指定起点，采用常见的 1、1、2、3、5……，而不是以 0 开始的另一种编号约定。

2. new int[30] 创建 30 个位置，下标为 0—29。因此“第 1 项”存入 f[0]，“第 30 项”存入 f[29]。

3. 先设置 f[0]=1、f[1]=1，这是递推必须拥有的两个初值。

4. 从 i=2 开始计算 f[i]=f[i-1]+f[i-2]。此时两个被引用的位置都已计算过，不会访问负下标。

5. 计算完所有元素后，再用第二个循环统一输出。分开“计算”和“输出”，有助于检查数组内部状态。

6. 例如 f[2]=1+1=2，f[3]=2+1=3。第 30 项为 832040，可作为检查项。

7. 前 30 项都在 int 范围内；若直接扩大项数，从第 47 项起就会超出 int，需要换成 long，并进一步考虑它的上限。

完整参考代码 · Main.java

```
public class Main {
    public static void main(String[] args) {
        int[] f = new int[30];
        f[0] = 1;
        f[1] = 1;
        for (int i = 2; i < f.length; i++) {
            f[i] = f[i - 1] + f[i - 2];
        }
        for (int i = 0; i < f.length; i++) {
            if (i > 0) System.out.print(" ");
            System.out.print(f[i]);
        }
        System.out.println();
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181 6765 10946 17711 28657 46368 75025 121393
    196418 317811 514229 832040
```

容易出错的地方

1. 循环不能从 i=0 就访问 f[i-2]，那样会出现负下标。

2. 若题目采用 0、1 起始，应同步修改初值与项号说明。

自己动手

1. 改为只输出第 30 项。

2. 输入 n（2<=n<=46），创建对应长度的数组并输出前 n 项。

第 22 题 20个数的最小值

题目

从键盘输入 20 个数存入数组，然后输出最小值。本答案使用 double，允许输入普通整数和小数。

基础思路

先读满数组，再用第一个元素初始化 min，依次比较剩余元素。

先掌握这些知识

double数组、数组输入、最小值维护

一步一步理解

1. 创建长度为 20 的 double 数组，数组负责保存题目要求的所有输入，而不是只在读入时临时比较。

2. 第一个循环从 0 到 19，每轮 nextDouble() 读取一个数放入对应位置，输入时用空格或换行分隔即可。

3. 读完后把 a[0] 赋给 min，保证候选最小值来自实际数据。不能把 min 随意设置成 0，否则全部为正数时会出错。

4. 第二个循环从下标 1 开始。只要 a[i]<min，就更新 min；否则保留原来的最小值。

5. 循环结束时，每个元素都参与过比较，min 就是全数组最小值；重复的最小值也不影响结果。

6. 使用 println 输出 double，因此整数输入 1 的最小值会显示为 1.0，这是类型的正常显示方式。

7. 样例按降序输入 20 到 1，可以检查程序是否会持续更新最小值，而不是只比较一次。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double[] a = new double[20];
        for (int i = 0; i < a.length; i++) a[i] = sc.nextDouble();
        double min = a[0];
        for (int i = 1; i < a.length; i++) {
            if (a[i] < min) min = a[i];
        }
        System.out.println(min);
    }
}
```

样例输入

```
20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1
```

样例输出

```
1.0
```

容易出错的地方

1. 本题固定输入 20 个数据，开头不需要额外输入一个 n。

2. new double[20] 的有效下标为 0—19。

自己动手

1. 输出最小值第一次出现的下标。

2. 同时求最大值与平均值。

第 23 题 方法返回课程平均分

题目

编写 fun 方法计算 n 门课程的平均分，并作为返回值返回。输入先给 n，再给 n 个成绩，输出平均分并保留两位小数。

基础思路

主方法读入成绩数组，把数组交给 fun；fun 求和后除以数组长度。

先掌握这些知识

数组参数、double返回值、平均数、输入数量检查

一步一步理解

1. 输入的第一个整数 n 是课程数，后面恰好跟 n 个成绩。用 double 保存成绩，以支持 89.5 这样的分数。

2. n 必须为正，因为没有课程时平均值没有定义；主方法在创建数组和调用 fun 之前检查它。

3. fun(double[] scores) 表示方法接收一个 double 数组。它不再读键盘，而是处理主方法已经读好的数据。

4. 数组参数自带 length 属性，所以不用再传一个可能与真实长度不一致的 n。

5. sum 先初始化为 0，再逐个累加成绩；return sum/scores.length 把结果交回调用处，并结束 fun。

6. main 中 average=fun(scores) 接收返回值，然后负责输出。这与在 fun 内直接打印不同，返回值还能继续用于其他计算。

7. 样例总分 270，课程数 3，平均分 90.00。%.2f 只负责显示两位小数，不改变原始成绩数组。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static double fun(double[] scores) {
        double sum = 0;
        for (int i = 0; i < scores.length; i++) sum += scores[i];
        return sum / scores.length;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

Main.java（续）

```
        int n = sc.nextInt();
        if (n <= 0) {
            System.out.println("课程数必须大于0");
            return;
        }
        double[] scores = new double[n];
        for (int i = 0; i < n; i++) scores[i] = sc.nextDouble();
        double average = fun(scores);
        System.out.printf(java.util.Locale.US, "%.2f%n", average);
    }
}
```

样例输入

```
3
80 90 100
```

样例输出

```
90.00
```

容易出错的地方

1. 返回类型必须是 double，不能声明为 void 后又 return 一个数。
2. 若总分使用 int，直接 sum/n 会先做整数除法。

自己动手

1. 增加一个方法求最高分，并由 main 分别调用。
2. 只统计及格课程的平均分，注意一个及格分都没有时如何处理。

第 24 题 冒泡排序升序

题目

将数组 {5,4,3,2,1} 用冒泡排序升序排列后输出。原文 int a[]=new int[5]{…} 不是合法 Java 初始化语法；本答案改为 int[] a={5,4,3,2,1}。

基础思路

每趟比较相邻元素，左边较大就交换，一趟结束把当前最大值放到未排序部分的末尾。

先掌握这些知识

数组初始化、冒泡排序、嵌套循环、交换

一步一步理解

1. Java 可以写 int[] a={5,4,3,2,1}，也可以写 new int[]{5,4,3,2,1}。不能在带初值列表的同时写 new int[5]{…}，分隔符也必须是英文逗号。

2. 冒泡排序只比较相邻两个元素。如果 a[j]>a[j+1]，就通过 temp 交换，让较大的元素向右移动。

3. 第一趟从左到右比较，5 会依次与 4、3、2、1 交换，最终位于最右端，数组变为 {4,3,2,1,5}。

4. 最右边的 5 已经确定位置，第二趟不用再比较它。因此内层上界是 a.length-1-i，每过一趟就缩短一次。

5. 长度为 5 的数组最多需要 4 趟，所以外层条件是 i<a.length-1。

6. 内层会访问 j+1，因此 j 不能到达最后一个有效下标，否则 a[j+1] 会越界。

7. 完成四趟后输出 1 2 3 4 5。本题要求冒泡排序，不使用 Arrays.sort 来替代练习过程。

完整参考代码 · Main.java

```
public class Main {
    public static void main(String[] args) {
        int[] a = {5, 4, 3, 2, 1};
        for (int i = 0; i < a.length - 1; i++) {
            for (int j = 0; j < a.length - 1 - i; j++) {
                if (a[j] > a[j + 1]) {
                    int temp = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = temp;
                }
            }
        }
        for (int i = 0; i < a.length; i++) {
            if (i > 0) System.out.print(" ");
            System.out.print(a[i]);
        }
        System.out.println();
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
1 2 3 4 5
```

容易出错的地方

1. 只做一趟只能保证最大的数到最后，不能保证全部有序。

2. 升序交换条件是左边大于右边；改成小于会得到降序。

自己动手

1. 改为降序排序。

2. 每趟输出一次数组，观察“最大的数向右冒泡”的过程。

第 25 题 日期是当年第几天

题目

输入年、月、日，输出它是该年的第几天。按公历闰年规则处理，非法日期输出“日期不合法”。

基础思路

先确定各月天数，再把该月之前各月的天数加上当前日。

先掌握这些知识

月份天数数组、闰年判断、合法性检查、前缀求和

一步一步理解

1. days 保存 1—12 月的天数，但下标从 0 开始，所以二月是 days[1]，输入的 month 对应 days[month-1]。

2. 公历闰年条件：能被 400 整除，或者能被 4 整除但不能被 100 整除。&& 优先于 ||，代码仍加括号便于阅读。

3. 2024 能被 4 整除且不能被 100 整除，是闰年；1900 能被 100 整除但不能被 400 整除，不是闰年；2000 是闰年。

4. 闰年只需要把二月从 28 改为 29，其余月份不变。

5. 先检查年和月份，再读取 days[month-1] 检查日。若反过来，月份 13 会导致数组越界。

6. total 先等于本月日期 day，再把当前月之前所有整月天数累加。3 月 1 日需累加一月和二月，不再加三月整月。

7. 2024 年 3 月 1 日为 31+29+1=61；平年的同一天为 31+28+1=60。

8. 本题采用正年份和公历规则的基础模型，不讨论历史上不同地区更换历法的日期。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int year = sc.nextInt();
        int month = sc.nextInt();
        int day = sc.nextInt();
        int[] days = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
        boolean leap = year % 400 == 0 || (year % 4 == 0 && year % 100 != 0);
        if (leap) days[1] = 29;
        if (year <= 0 || month < 1 || month > 12) {
            System.out.println("日期不合法");
            return;
        }
        if (day < 1 || day > days[month - 1]) {
            System.out.println("日期不合法");
            return;
        }
```

Main.java（续）

```
        int total = day;
        for (int i = 0; i < month - 1; i++) total += days[i];
        System.out.println(total);
    }
}
```

样例输入

```
2024 3 1
```

样例输出

```
61
```

容易出错的地方

1. “能被 4 整除就是闰年”会错误处理 1900。
2. 累加时不能把当前月的全部天数也加进去。

自己动手

1. 增加输出这一年总共有 365 天还是 366 天。
2. 输入一年中的第 k 天，反向求对应的月和日。

第 26 题 有序数组二分查找

题目

编写方法，在已按从小到大排列的 n 个元素数组中二分查找 x，找到返回下标，否则返回 -1。输入 n，再输入 n 个有序整数，最后输入 x；下标从 0 开始。

基础思路

维护左右端点，比较中间元素，每次排除不可能的一半区间。

先掌握这些知识

有序数组、二分区间、方法返回、边界

一步一步理解

1. 二分查找的前提是数组已经升序排列。若输入无序，比较大小后就无法确定应该舍弃哪一半，算法不再可靠。
2. left 和 right 表示仍可能存在目标的闭区间，初始覆盖整个数组，即 0 到 length-1。
3. mid=left+(right-left)/2 取中间下标；比直接 (left+right)/2 更不容易发生加法溢出。
4. a[mid]==x 时立即返回 mid。如果 a[mid]<x，左半边所有数都不够大，于是令 left=mid+1；否则令 right=mid-1。

5. 必须排除已经比较过的 mid，因此有 +1 和 -1。这样区间会不断缩小，不会停留在原地形成死循环。

6. left<=right 时哪怕只剩一个元素也要检查；left>right 时区间为空，目标不存在，返回 -1。

7. 样例中 7 是第 4 个元素，但 Java 下标从 0 开始，所以返回 3。空数组 n=0 直接返回 -1。

8. 若有多个相等元素，普通二分返回其中某一个下标，不保证第一个；题目只要求查找成功，不要求首个位置。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static int binarySearch(int[] a, int x) {
        int left = 0;
        int right = a.length - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (a[mid] == x) return mid;
            if (a[mid] < x) left = mid + 1;
            else right = mid - 1;
        }
        return -1;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if (n < 0) {
            System.out.println("n不能小于0");
            return;
        }
        int[] a = new int[n];
        for (int i = 0; i < n; i++) a[i] = sc.nextInt();
        int x = sc.nextInt();
        System.out.println(binarySearch(a, x));
    }
}
```

样例输入

```
5
1 3 5 7 9
7
```

样例输出

```
3
```

容易出错的地方

1. 有序是必需条件；不要把数组排序后又把新下标当作原数组下标。

2. 更新时写 left=mid 或 right=mid 可能使区间无法缩小。

自己动手

1. 分别查找数组最左端和最右端的数。

2. 修改算法，使有重复元素时返回第一次出现的下标。

第 27 题 矩阵最大最小值及位置

题目

输入一个 3×4 整数矩阵。原题正文要求最大值，末尾括号又写“最小值”，故同时给出最大值、最小值及各自所在行列。行列从 1 开始；重复极值报告按行扫描遇到的第一个位置。

基础思路

将第一个元素作为最大最小初值，逐行扫描，同时维护数值和所在下标。

先掌握这些知识

二维数组、嵌套循环、极值与位置记录

一步一步理解

1. int[][] 表示二维数组，a[i][j] 的第一个下标 i 是行，第二个下标 j 是列；这里有 3 行、4 列。

2. 输入循环外层控制行，内层控制列，共读入 12 个整数。分成三行输入只是为了便于观察，不影响 Scanner 读取。

3. max 和 min 都从 a[0][0] 开始，不能随意设为 0；若矩阵全负，max=0 就会得到不在矩阵中的错误答案。

4. 每个极值都配两个位置变量。发现更大的数时，要同时更新 max、maxRow、maxCol，不能只改数值。最小值同理。

5. 比较使用严格的大于和小于，所以相等值不会覆盖已记录的位置。这使重复极值报告逐行扫描遇到的第一个位置。

6. 数组下标从 0 开始，但题目要报告行号列号，本资料按日常习惯从 1 开始，因此输出时加 1。

7. 原文“最大元素”后又括注“最小值”，存在歧义。本答案同时给出两者，考试若只要求其中之一，保留对应输出即可。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[][] a = new int[3][4];
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 4; j++) a[i][j] = sc.nextInt();
        }
        int max = a[0][0], min = a[0][0];
        int maxRow = 0, maxCol = 0, minRow = 0, minCol = 0;
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 4; j++) {
                if (a[i][j] > max) {
                    max = a[i][j]; maxRow = i; maxCol = j;
                }
                if (a[i][j] < min) {
```

Main.java（续）

```
                    min = a[i][j]; minRow = i; minCol = j;
                }
            }
        }
        System.out.println("最大值=" + max + " 行=" + (maxRow + 1) + " 列=" + (maxCol + 1));
        System.out.println("最小值=" + min + " 行=" + (minRow + 1) + " 列=" + (minCol + 1));
    }
}
```

样例输入

```
1 2 3 4
5 6 7 8
9 10 11 12
```

样例输出

```
最大值=12 行=3 列=4
最小值=1 行=1 列=1
```

容易出错的地方

1. 不要把 a[i][j] 的行列顺序写反。
2. 字符串拼接中要给 maxRow+1 加括号，否则可能拼成 01，而不是先算出 1。

自己动手

1. 输出所有出现最大值的位置，而不只第一个。
2. 把行数和列数改为由用户输入。

第 28 题 数组顺序查找

题目

编写方法，在 n 个元素的数组中查找 x；找到返回下标，否则返回 -1。输入 n、n 个整数、x。数组不必有序；若重复出现，返回第一次出现的下标。

基础思路

从下标 0 开始逐个比较，遇到相等元素立即返回；全部检查后仍未找到则返回 -1。

先掌握这些知识

顺序查找、数组参数、提前返回

一步一步理解

1. 与第 26 题不同，本题没有要求数组有序，因此最直接可靠的方法是顺序查找。
2. search 接收数组 a 和目标 x，从第一个位置开始比较 a[i]==x。

3. 一旦相等就 return i，结束整个 search 方法。因为按下标递增扫描，所以返回的是第一次出现的位置。

4. return -1 必须放在整个循环之后。只有检查完所有元素仍不相等，才有资格断定不存在。

5. 样例中的 4 位于第 4 个位置，对应 Java 下标 3，所以输出 3；输出的不是元素值 4。

6. n=0 时创建空数组，循环一次也不执行，直接返回 -1，不访问 a[0]，因此不会越界。

7. 最坏情况下要比较 n 次，复杂度 O(n)；二分查找比较次数少，但要求有序，不能直接套在无序输入上。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static int search(int[] a, int x) {
        for (int i = 0; i < a.length; i++) {
            if (a[i] == x) return i;
        }
        return -1;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if (n < 0) {
            System.out.println("n不能小于0");
            return;
        }
        int[] a = new int[n];
        for (int i = 0; i < n; i++) a[i] = sc.nextInt();
        int x = sc.nextInt();
        System.out.println(search(a, x));
    }
}
```

样例输入

```
5
8 3 9 4 1
4
```

样例输出

```
3
```

容易出错的地方

1. 在 if 的 else 中直接 return -1 会只比较第一个元素。

2. 不能使用 = 代替 == 判断两个整数是否相等。

自己动手

1. 改为输出 x 出现的所有下标。

2. 增加一个计数方法，统计 x 出现了几次。

第 29 题 四位回文数

题目

输入一个四位正整数，判断它是不是回文数，例如 1221 是，1234 不是。不是 1000—9999 的输入输出“请输入四位正整数”。

基础思路

四位数只需判断千位等于个位，同时百位等于十位。

先掌握这些知识

整数位数拆分、逻辑与、回文判断

一步一步理解

1. 回文要求从两端向中间对应位置都相同。四位数只有两组需要比较：第 1 位与第 4 位，第 2 位与第 3 位。

2. 先限制 1000<=n<=9999。作为整数输入的 0123 只有数值 123，不是一个四位正整数。

3. n/1000 取千位；n/100%10 取百位；n/10%10 取十位；n%10 取个位。整数除法负责去掉右侧位数。

4. 用两个 == 分别比较外侧、内侧数字，再用 && 连接，必须两组都相等才能判为回文。

5. 1221 拆为 1、2、2、1，两组都相等；1234 的千位与个位已不相等，所以不是回文。

6. 1001 也是回文，内部的两个 0 不会造成特殊问题。这种位数拆分法无需数组、字符串反转或复杂方法。

7. 本题只针对四位数。若要判断任意位数，可以学习第 6 题的逐位逆序，或用字符串比较首尾。

完整参考代码 · Main.java

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if (n < 1000 || n > 9999) {
            System.out.println("请输入四位正整数");
            return;
        }
        int thousands = n / 1000;
        int hundreds = n / 100 % 10;
        int tens = n / 10 % 10;
        int units = n % 10;
        if (thousands == units && hundreds == tens) {
            System.out.println("是回文数");
        } else {
            System.out.println("不是回文数");
        }
    }
}
```

样例输入

```
1221
```

样例输出

```
是回文数
```

容易出错的地方

1. 只比较千位与个位不够，例如 1231 并不是回文。
2. 数值 0000 等于 0，不属于题目要求的四位正整数。

自己动手

1. 改成判断五位回文数，指出中间那一位为什么不用比较。
2. 用字符串方式比较首尾，扩展到任意长度。

第 30 题 百钱百鸡枚举

题目

小鸡 1 元买 3 只，公鸡 3 元一只，母鸡 5 元一只。用 100 元恰好买 100 只鸡，列出所有方案。各类数量允许为 0；若另要求每种都必须购买，应排除母鸡为 0 的方案。

基础思路

枚举公鸡和母鸡数量，用总只数确定小鸡，再验证小鸡能否三只成组以及总价是否为100。

先掌握这些知识

枚举、约束方程、整数价格、嵌套循环

一步一步理解

1. 设公鸡、母鸡、小鸡数量分别为 rooster、hen、chick，都必须是非负整数。总只数条件为 rooster+hen+chick=100。
2. 100 元最多买 33 只公鸡，最多买 20 只母鸡，所以两个循环只需分别枚举 0—33 和 0—20。
3. 每确定一组公鸡、母鸡数量，小鸡数量就由 100-rooster-hen 唯一决定，因此不必再套第三重循环。
4. 小鸡 3 只 1 元，其费用为 chick/3。必须先检查 chick%3==0，才能使用整数除法而不截断真实费用。
5. 同时检查总费用 3*rooster+5*hen+chick/3==100，满足数量和费用两条约束才是一组有效方案。
6. 例如公鸡 4、母鸡 12、小鸡 84，共 100 只，费用为 12+60+28=100 元。
7. 全部方案共 4 组，最后一组为 25、0、75。如果题目额外规定三种鸡至少各一只，则只保留前三组；原文没有这一附加条件。
8. 也可独立验算：把价格方程乘 3，再减去只数方程，得到 4*公鸡+7*母鸡=100，四组输出均满足这个关系。

完整参考代码 · Main.java

```
public class Main {
    public static void main(String[] args) {
        int count = 0;
        for (int rooster = 0; rooster <= 100 / 3; rooster++) {
            for (int hen = 0; hen <= 100 / 5; hen++) {
                int chick = 100 - rooster - hen;
                if (chick >= 0 && chick % 3 == 0
                        && 3 * rooster + 5 * hen + chick / 3 == 100) {
                    System.out.println("公鸡=" + rooster + " 母鸡=" + hen + " 小鸡=" + chick);
                    count++;
                }
            }
        }
        System.out.println("方案数=" + count);
    }
}
```

样例输入

```
（无输入，直接运行）
```

样例输出

```
公鸡=4 母鸡=12 小鸡=84
公鸡=11 母鸡=8 小鸡=81
公鸡=18 母鸡=4 小鸡=78
公鸡=25 母鸡=0 小鸡=75
方案数=4
```

容易出错的地方

1. 整数表达式 1/3 等于 0，不能写 chick*(1/3) 计算费用。
2. 不要把题目给定的公鸡 3 元、母鸡 5 元与其他版本百钱百鸡的价格混淆。

自己动手

1. 修改条件，让三种鸡都至少买一只。
2. 把预算和总只数改为输入，观察哪些输入没有方案。

复习与参考

复习时先做没有输入的题，再做单次输入，最后做循环输入和多个对象的题。遇到相同类名，回到对应题目录单独编译。

原题库：用户提供的阶段一、阶段二题库 PDF。原题中的重复编号、示例编号和歧义已经在封面说明或对应题目中标出。

Oracle Java Tutorials：语言基础

https://docs.oracle.com/javase/tutorial/java/nutsandbolts/index.html

Oracle Java Tutorials：面向对象概念

https://docs.oracle.com/javase/tutorial/java/concepts/index.html

本文使用 JDK 17 可运行的基础语法；上面的概念教程版本较早，变量、循环、类与继承等基础仍适用。